
Progress Report: A Transparent MoE Architecture for Sequence-Level Routing

Björn Pettersson^{1 2} Hosung Lee^{1 2} Khue^{1 2}

1. Introduction

While the widely used token level routing within mixture of experts MoE has been shown to be an effective way to scale up large language models, while keeping inference costs down. The MoE architecture itself could hold interesting properties for interpretability if expert specialisation could be domain specific, rather than based on token properties, letting us see in real time what experts are used to answer different types of prompts based on routing behaviour. However this type of analysis becomes useful to the extent that experts actually become topic experts. Projects like Steer Moe (Feng et al., 2025) show that analysis of routing patterns and dynamic altering of them in regards to alignment can be an effective way of changing model behaviour during inference, strengthening the argument around why expert domain interpretability might provide useful properties for this type of model behaviour adjustments. Clear domain expert specialisation could also show interesting properties in terms of model modularity and for mitigating catastrophic forgetting (Gururangan et al., 2021).

Gururangan et al also show great potential for hard routing based on document content. However this approach heavily relies on labelled domain specific data, which is a major limitation for LLMs often trained on large unlabelled datasets such as openWebText.

Fan et al (Fan et al., 2024) shows that sequence level routing organically gives rise to a weak form of this human like domain expertise, but not to an extent that would make this type of routing behaviour interesting for an alignment point of view or as strong as seen by Gururangan et al.

We propose a middle ground where pretraining or weight upcycling of narrowly trained domain experts are combined in the MoE and then continuously trained on unstructured data. Our hypothesis is that initialisation with pretrained weights may strengthen the weak organic domain specialisation discovered by fan et al and hence provide a step towards clearer more interpretable and scalable domain specialisation in MoE architectures.

2. Related Works

The list of reviewed papers is shown on Table 1.

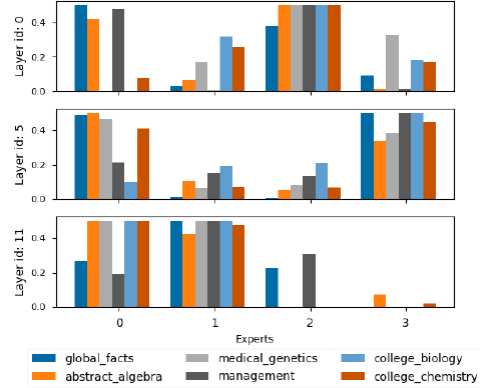


Figure 1. From (Fan et al., 2024) showing weak expert specialization organically emerging from sequence level routing

- (Fan et al., 2024) : Systematically explore how variations in the number of experts, gating mechanisms, and sparsity levels affect the model's performance and efficiency.
- (Sukhbaatar et al., 2024) : Introduce Branch-Train-MiX to combine multiple expert models with a MoE system, enabling the efficient training and utilization of specialized experts while maintaining the scalability and flexibility of the MoE architecture.
- (Gururangan et al., 2021) : Introduce DEMIX layers that allow for better specialization of models by domain, improving performance in complex, multi-domain tasks.
- (Nakamura et al., 2025) : Introduce a technique for training sparse MoE models named Drop-Upcycling, focusing on partial re-initialization to overcome training instability in sparse MoE systems.
- (Feng et al., 2025) : Propose Steer-MoE method that improves efficiency and performance of tasks requiring audio-language integration in a MoE framework.

Table 1. Paper Review on MoE Specialization and Routing Unit

PAPERS	ROUTING UNIT
FAN ET AL. (2024)	TOKEN/SEQUENCE
SUKHBAATAR ET AL. (2024)	TOKEN (TOP-2)
GURURANGAN ET AL. (2021)	SEQUENCE
NAKAMURA ET AL. (2025)	TOKEN (TOP-2)
FENG ET AL. (2025)	TOKEN

3. Method

3.1. The Original Experiment

The experiment by (Fan et al., 2024) originally showing weak domain specialization when trained on sequence level data was based on the base architecture of GPT2-mini, from the nanoGPT repository with a LoRA extension. We aim to reproduce a scaled down version of the result in Figure 2, where slightly different activation patterns based on the subject pointed towards weak specialization (figure 1). The figure was obtained by:

1. Training a GPT2-mini based transformer on openWebText, a general dataset.
2. Giving it subject specific tasks from MMLU a test dataset containing subject specific questions and answers.
3. Recording routing behaviour when the MoE was faced with these subject specific tasks. We have 4 experts in each of our 3 transformer layers. This is what we see in the figure. (Fig 2)
4. For non-specialisation, we would expect equal use of each expert based on subject. However, here we see some different staple heights suggesting the MoE chooses to route differently based on subject. This is what the authors describe as weak specialisation.

Fan et al. (Fan et al., 2024) trained on a single A100-SXM4-40GB GPU, with the following modifications to the nanoGPT architecture. For our specific figure a top k=2 level routing was used. For the sequence level routing the softmax function was applied twice as below:

$$p_i(x) = \frac{e^{h_i(x)}}{\sum_j e^{h_j(x)}}, \quad y = \sum_{i=\tau} \frac{e^{p_i(x)}}{\sum_{j \in \tau} e^{p_j(x)}} E_i(x)$$

Here:

- x - is the input token or sequence embedding
- τ is the set of top-k indices ($K = 2$)

Table 2. Original Fan et al. Experiment Parameters

Parameter	Value
Base model	nanoGPT w/ LoRA
Dropout	0.2
Learning Rate (LR)	9.6×10^{-4}
LR (Minimum)	9.6×10^{-5}
Weight Decay	0.5
Biases Enabled	6k iterations
Tokens / Iteration	1.05M
Gradient Accumulation	128
Batch Size	8
Sequence Length	1024
Total Tokens Seen	6B
No. of Experts (N)	4
Tokens / Parameter	20 (Chinchilla)
Load Balance Loss (λ)	0.01
Tokenizer	OpenAI GPT-2

- $p_i(x)$ are the logits produced by the gating network as shown
- $E_i(x)$ is the output of the i -th expert
- y is the overall output, which is a weighted sum of the two selected experts.

3.2. Scaling down Fan et al.

For our experiment we propose a smaller version of Fan et al. to attempt reproducing weak topic specialisation for sequence level routing, training on a subset of openWebText to be able to run on a RTX 5080 GPU with 16GB VRAM as opposed to the A100- SXM4-40GB GPU used in the original experiment. Figure 2 by Fan et al seems to suggest that routing behaviour stabilize quite early in the training process possibly allowing us to get away with fewer iterations, here 3000 instead of 6000, when only looking at routing behaviour.

We start from the same basic nanoGPT architecture based on Daniel Grittnner’s repository a transformer with 12 blocks and a LoRA extension, adding a `SequenceMoE` class alongside the basic `MLP` class used when the number of experts is set to more than 0. This class contains a forward function for the MoE, which also handles load balancing with a parameter λ . We saw in Figure 2 that the load balancing had a positive effect on making sure experts stabilize early, which is a useful assumption for scaling down this experiment. Load balancing mainly aims to mitigate expert collapse. Adding load balancing we update our loss as follows:

$$L_{total} = L_{LM} + \lambda L_{aux}$$

The original language modelling loss, cross entropy term L_{LM} , is measuring how well the model predicts the next

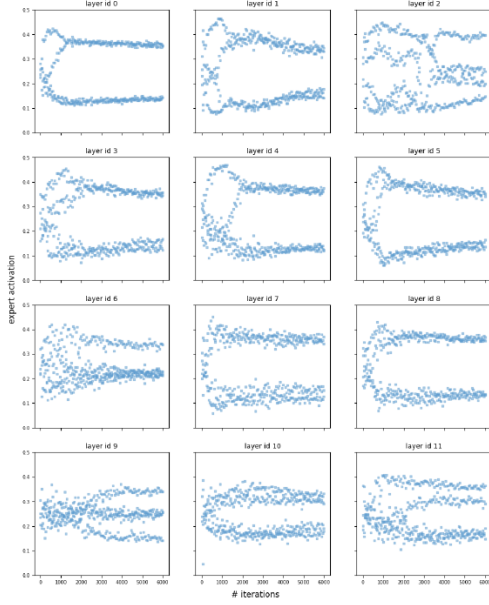


Figure 7: Expert activations from Layer-wise Sequence-level Top-2 routing when load balancing loss is applied.

Figure 2. (Fan et al., 2024) showing how routing behaviour often stabilize early in training, here $k=2$ $n=4$, with load balancing

token in a sequence:

$$L_{LM} = -\frac{1}{T} \sum_{t=1}^T \log(P(y_t | x_{1:t-1}))$$

Where $P(y_t | x_{1:t-1})$ is the probability assigned by the model to the correct next token y_t , and T is the total number of tokens being predicted (the sequence length). In the GPT class Forward function this is found as `main_loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)`.

Where L_{aux} is a product of expert importance/traffic (P) and expert load/frequency of selection (F) summed over all experts (N):

$$L_{aux} = N \sum_{i=1}^N P_i F_i$$

Expert importance P_i , is the expected fraction of traffic routed to expert i . The average of the softmax probabilities $p_i(x)$ assigned to expert i across the entire batch B :

$$P_i = \frac{1}{|B|} \sum_{x \in B} p_i(x)$$

Expert Load (F_i) measures the actual fraction of sequences that selected expert i . It is the average of a binary indicator function ($1_{selected}$) across the batch B , which is 1 if expert

i was one of the top K chosen experts for sequence x and 0 otherwise:

$$F_i = \frac{1}{|B|} \sum_{x \in B} 1_{selected}(x, i)$$

In our function `SequenceMoE.forward` we have P_i as expert-importance, F_i as expert-load, and L_{aux} as load-balancing-loss, and L_{total} as total-loss.

3.2.1. THE MODIFIED FORWARD FUNCTION:

- We feed in a batch x , which contains B samples of T tokens with embedding dimension C .
- We then average over the dimension T , to get a batch of B average tokens of embedding dimension C .
- Our router assigns weights based on these average tokens. We get these by passing it through our dense FFN router layer to obtain the logits, which are then normalized with softmax to obtain the routing probabilities for each average token (the first part of the “double softmax”).
- We use the `topk` function to obtain the selected top k experts and their routing weighting/probability. We then normalize this routing probability over the k selected experts so it sums to 1 (the second part of the “double softmax”).
- We then loop through the batch B times, and obtain, this time the actual sequence (T, C).
- In an inner loop, we loop through each of our selected experts (from earlier based on our average tokens) but now we route the actual sequences.
- For each sequence we then have a weighted average between the output from the top k experts (`output_i`).
- We gather all the weighted average logits from passing the sequences through the expert FFNs (all `output_is`) into a final output matrix, this is the updated x vector after passing it through the MoE layer. This vector is later passed through a language modelling head (see the GPT class), a linear layer to project the x vector from the embedding dimension C up to the vocabulary size. We then take the softmax over this final vector and choose the next token with the highest probability.

3.3. Optimisation

Since running the above proposed code literally is computationally expensive and we are limited to a 16gb gpu,

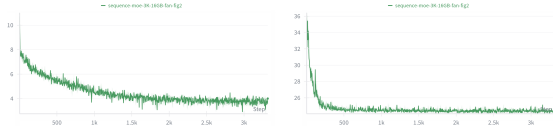


Figure 3. LM Loss (left), AUX Loss (right)

we implemented an optimisation. The code is almost functionally the same: a Sequence-level Mixture-of-Experts that (1) routes a sequence to top-k experts, (2) runs experts on the full sequence, (3) aggregates weighted expert outputs, and (4) returns an auxiliary load-balancing loss. For our purposes we deem this tradeoff close enough and not detrimental to our results.

The general optimisation of model.py vectorizes the expert computation by bucketing sequences per expert and running each expert once on its assigned mini-batch, while moe-model.py runs experts per-sequence (nested loop over batch then top-k), causing repeated small forward passes. This vectorization reduces duplicated work and greatly improves throughput on GPUs when multiple sequences share experts.

4. Base Line Model Experiments

The experiment in our local setting was set according to Table 2. It took about 14 hours to train 3k iterations, and the results are shown in Figure 3. As illustrated in the loss curves, the LM loss shows a consistent downward trend, indicating steady convergence and effective learning of the dataset. Meanwhile, the auxiliary loss (load balancing loss) decreased rapidly in the early stages and stabilized, confirming that the router was successfully optimized for equal expert utilization. This is further shown by the expert routing visualization (Figure 4), where the selection frequency for all four experts across all 12 layers are tightly bounded around 0.5 and under. However, we can also check that distinct expert specialization may not have fully emerged yet in our training setting.

5. Future Plans

- ✓ **Baseline Setup:** Train a scaled-down Mixture-of-Experts (MoE) model similar to Fan et al.
- ✗ **Baseline Evaluation:** Evaluate this model on the MMLU benchmark, ensuring it reproduces weak expert specialization based on sequence-level routing.
- ✗ **Modified Model Training:**
 - ✗ **3.1 Narrow Pre-training:** Modify the baseline by first narrowly pre-training the individual experts on the **MMLU auxiliary train dataset** (which is domain-specific).

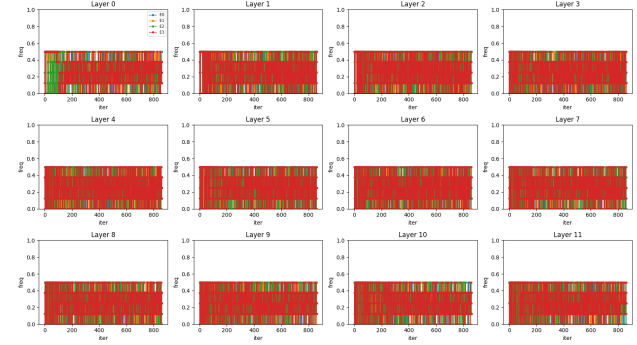


Figure 4. Expert Activation Frequency

– ✗ **3.2 Joint Training:** Transfer the weights into the broader MoE architecture and run a joint MoE training phase on the **OpenWebText** dataset.

- ✗ **Modified Model Evaluation:** Evaluate the routing behavior and overall performance on MMLU.
- ✗ **Analysis:** Observe if the routing behavior is significantly different from the baseline and in what way. The overall aim is to answer if the narrow pre-training step has an effect on expert domain specialization.

Table 3. Experiment Parameters (Modified for 16GB VRAM)

Parameter	Value
Base model	GPT-2 Small (MoE)
Dropout	0.2
Learning Rate (LR)	9.6×10^{-4}
LR(Minimum)	9.6×10^{-5}
Weight Decay	0.5
Bias	True
Tokens / Iteration	65,536
Gradient Accumulation	32
Batch Size	4
Sequence Length	512
Total Tokens Seen	$\approx 196M$
Max Iterations	3000
No. of Experts (N)	4
Routed Experts (K)	2
Load Balance Loss (λ)	0.01
Tokenizer	GPT-2 (OpenWebText)

References

- Fan, D., Messmer, B., and Jaggi, M. Towards an empirical understanding of moe design choices. arXiv preprint arXiv:2402.13089, 2024.
- Feng, R., Zhang, B., Liang, S., and Yuan, Z. Steer-moe: Efficient audio-language alignment with a mixture-of-experts steering module. arXiv preprint arXiv:2510.13558, 2025.

Gururangan, S., Lewis, M., Holtzman, A., Smith, N. A., and Zettlemoyer, L. Demix layers: Disentangling domains for modular language modeling. arXiv preprint arXiv:2108.05036, 2021.

Nakamura, T., Akiba, T., Fujii, K., Oda, Y., Yokota, R., and Suzuki, J. Drop-upcycling: Training sparse mixture of experts with partial re-initialization. arXiv preprint arXiv:2502.19261, 2025.

Sukhbaatar, S. et al. Branch-train-mix: Mixing expert llms into a mixture-of-experts llm. arXiv preprint arXiv:2403.07816, 2024.

A. Appendix.